akhil_report

by Akhil Akhil

Submission date: 28-Apr-2026 07:07PM (UTC+0530)
Submission ID: 2946613041
File name: capstone_report__1.docx (3.04M)
Word count: 9160
Character count: 47671

CAPSTONE PROJECT REPORT

(Project Term September 2025 – May 2026)

Smart Learning Ecosystem Using Generative AI for Personalized and Gamified Education

Submitted by:

Sr. No.	Name of Student	Registration Number
1	Neelapu Venkata Naidu	12213327
2	Nithin Yalakala	12205244
3	Akhil Mudadla	12212650
4	Dileep Lopinti	12204945

Project Group Number: 2RGC0632
Course Code: CSE439

Under the Guidance of:

Dr. Alok Misra
Professor
Devops
School of Computing and Artificial Intelligence
Lovely Professional University

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
LOVELY PROFESSIONAL UNIVERSITY
Phagwara, Punjab – 144411

School of Computer Science and Engineering



LOVELY PROFESSIONAL UNIVERSITY

TOPIC APPROVAL PERFORMA

School of Computer Science and Engineering

Program : P132::B.Tech. (Computer Science and Engineering)

COURSE CODE : CSE439 **REGULAR/BACKLOG :** Regular **GROUP NUMBER :** 2RGC0632

Supervisor Name : Dr. Alok Misra **UID :** 31011 **Designation :** Professor

Qualification : _____ **Research Experience :** _____

SR.NO.	NAME OF STUDENT	Prov. Regd. No.	BATCH	SECTION
1	Neelapu Venkata Naidu	12213327	2022	422YH
2	Akhil Mudadla	12212650	2022	222EK
3	M R Bilva Sri	12212545	2022	422GC
4	Yalakala Nithin	12205244	2022	322QB
5	Dileep Lopinti	12204945	2022	222ST

SPECIALIZATION AREA : Artificial Intelligence and Machine Learning **Supervisor Signature:** _____

PROPOSED TOPIC : AI Learning Path Generator for Personalized Tech Roadmaps

Qualitative Assessment of Proposed Topic by PAC		
Sr.No.	**Parameter**	**Rating (out of 10)**
1	Project Novelty: Potential of the project to create new knowledge	7.00
2	Project Feasibility: Project can be timely carried out in-house with low-cost and available resources in the University by the students.	7.00
3	Project Academic Inputs: Project topic is relevant and makes extensive use of academic inputs in UG program and serves as a culminating effort for core study area of the degree program.	7.00
4	Project Supervision: Project supervisor's is technically competent to guide students, resolve any issues, and impart necessary skills.	7.00
5	Social Applicability: Project work intends to solve a practical problem.	6.80
6	Future Scope: Project has potential to become basis of future research work, publication or patent.	6.80

PAC Committee Members		
PAC Member (HOD/Chairperson) Name: Dr. Harjeet Kaur	UID: 12427	Recommended (Y/N): Yes
PAC Member (Allied) Name: Sudha Shanker Prasad	UID: 21826	Recommended (Y/N): Yes
PAC Member 3 Name: Cherry Khosla	UID: 13436	Recommended (Y/N): NO

Final Topic Approved by PAC: **AI Learning Path Generator for Personalized Tech Roadmaps**

Overall Remarks: Approved

PAC CHAIRPERSON Name: 11265::Dr. Dalwinder Singh **Approval Date:** 05 Dec 2025

DECLARATION

We, the undersigned students of B.Tech (Computer Science and Engineering), Lovely Professional University, hereby declare that the Capstone Project – II report titled "Smart Learning Ecosystem Using Generative AI for Personalized and Gamified Education" submitted in partial fulfilment of the requirements for the Bachelor of Technology degree is our own original work.

We also declare that:

1. This report has not been submitted for any other degree or examination anywhere else.
2. All the data, results, and conclusions presented here are based on our own work unless a reference has been clearly mentioned.
3. We have properly cited all sources and references used in this report.
4. AI writing tools were used only for minor language corrections. All the technical content, design choices, and implementation work in this project are entirely our own.

Project Group Number: 2RGC0632

Sr. No.	Name of Student	Registration Number	Signature
1	Neelapu Venkata Naidu	12213327	
2	Nithin Yalakala	12205244	
3	Akhil Mudadla	12212650	
4	Dileep Lopinti	12204945	

Date: _____

Place: _____

CERTIFICATE

This is to certify that the Capstone Project – II report entitled "Smart Learning Ecosystem Using Generative AI for Personalized and Gamified Education" submitted by the group of students mentioned below, from the Department of Computer Science and Engineering, Lovely Professional University, is a record of genuine work carried out under my guidance and supervision. The project meets the university requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering.

Course Code: CSE439

Section: 2C632

_____

(Signature of Supervisor / Guide)

Dr. Alok Misra
Professor

Devops

School of Computing and Artificial Intelligence
Lovely Professional University

Date: _____

ACKNOWLEDGEMENT

We would like to thank Dr. Alok Misra, Assistant Professor, Department of Computer Science and Engineering, Lovely Professional University, for guiding us throughout this project. His suggestions during the planning phase helped us set a realistic scope and avoid going off track. Whenever we were stuck on a design decision, his feedback gave us a clearer direction.

We also thank Lovely Professional University for the academic environment and the resources provided. Access to journals and development tools through the university helped us during both the research and the building phases of this project.

Thanks are also due to the open-source communities behind Next.js, Google Gemini API, MongoDB, CodeMirror, and Tailwind CSS. These tools saved us a lot of time and made the implementation feasible within one semester.

Lastly, we want to thank the 50 students who agreed to use the platform during the evaluation period. Their feedback was direct and useful, and a lot of small improvements in the UI came directly from what they reported.

The Group

Date: _____

ABSTRACT

A major issue with on-line learning of programming is that there is too much content and no direction [14]. They begin three courses, complete zero and are more confused than before. This is sometimes referred to as "tutorial hell". The content is there, but they don't tell you what to do today, what to do tomorrow and whether you have understood what you have just read.

Our project tries to address this issue by creating a platform that provides a personalised learning plan on a day-to-day basis for any topic that a user is interested in, using the Google Gemini 2.5-Flash generative AI model [8]. The user specifies what they want to learn, their level of experience and the number of days they have. The AI creates a learning plan and the system takes them through it day-by-day, presenting theory, a programming exercise (with a code editor), and a 15-question quiz [1], [2].

The system uses Next.js 15 (with the App Router) [9], MongoDB Atlas to store data [10], Google Gemini for the AI content [8], CodeMirror 6 for the in-browser code editor [11], and Recharts for the progress dashboard [19]. Zod is used to validate the JSON returned from the AI so it doesn't get to the frontend. JWT & bcrypt are used for authentication.

We also use gamification in the form of streaks and badges [6], [3]. A streak is the number of consecutive days the user has a learning session. Badges are earned at the end of the first day, a 7-day streak, and consistently scoring 80% or higher. A personalised mentor chatbot, again powered by Gemini [8] is also available on each day of learning to answer context-specific questions [16].

The platform achieved a 68% success rate (of those who completed a roadmap) in a two-week study involving 50 participants, compared with much lower success rates reported by participants on other platforms. The active learning time per day was 40 minutes. The generation of roadmap JSON was successful on the first or second try 93% of the time. While not spectacular, these results are realistic and indicate the basic idea is working.

The main limitations at this stage are the lack of server-side Python execution, the absence of cross-device session sync, and API cost scaling challenges. These are documented clearly in the later chapters.

Keywords: *Generative AI, Adaptive Learning, Next.js, Gemini API, Personalized Education, Gamification, CodeMirror, Quiz-Based Learning*

TABLE OF CONTENTS

S. No.	Section Title	Page No.
	Abstract	vi
	List of Figures	viii
	List of Tables	viii
1	Introduction	1
1.1	Background and Motivation	1
1.2	Terminology	2
1.3	Problem Statement	3
1.4	Objectives	3
1.5	Scope and Limitations	4
1.6	Expected Outcomes	5
2	Rationale and Scope of the Study	6
3	Existing System	7
4	Problem Analysis	10
5	Software Requirement Analysis	13
6	Design	16
7	Testing	24
8	Implementation	29
9	Project Legacy	37
10	User Manual	40
11	System Snapshots	45
12	References	50

LIST OF FIGURES

Figure No.	Figure Title	Page
Figure 1	Level-0 DFD – Current Online Learning Flow	8
Figure 2	Smart Learning Ecosystem – Full System Architecture	9
Figure 3	Research Methodology Flowchart	19
Figure 4	Data Flow Diagram – Level 0	20
Figure 5	Data Flow Diagram – Level 1	21
Figure 7	Main Dashboard – Roadmaps, streak counter, badges, and progress chart	46
Figure 8	Daily Learning Page – Theory panel, CodeMirror editor, and quiz panel	47
Figure 9	Quiz Interface – Score display, correct/incorrect breakdown, and progression message	48
Figure 10	AI Mentor Chat Panel – Conversation with contextual roadmap awareness	49

LIST OF TABLES

Table No.	Table Title	Page
Table 1	Project Timeline and Work Plan	12
Table 2	Functional Requirements	14
Table 3	Non-Functional Requirements	15
Table 4	Technology Stack Summary	29
Table 5	User Acceptance Testing – Test Cases	25
Table 6	System-Level Performance Metrics	28
Table 7	LightGBM Configuration Parameters	32
Table 8	Expected vs. Actual Outcomes	38
Table 9	System Requirements	40
Table 10	Troubleshooting Guide	44

1. INTRODUCTION

1.1 Background and Motivation

The number of online courses has increased rapidly in recent years [14]. You can find a free or low-cost course on almost any programming topic. This seems like a win-win situation. It has become more difficult for students to self-learn than 5 years ago due to information overload [15].

If a learner searches for "how to learn Python" she will find dozens of courses, videos, blog posts and documentation. Some start from scratch, some require more familiarity with programming, some cover topics in a different sequence. They spend the first week trying to decide which one to use, start a couple of them, but never complete any. This is what the developer community refers to as "tutorial hell" - not because the tutorials are poor, but because without context, there is no guidance on what to do today, and how today's learning relates to tomorrow's work [2].

It gets worse over time. Even if a learner can choose one course, they get discouraged after a week or two because there is no way to see how they are progressing, no way to reward for their efforts and no way to make sure they understood the content before moving on to the next topic [7].

With large language models such as Google Gemini [8], it's now possible to create a detailed plan of day-by-day learning for any topic as the user is requesting it [4]. This is not course recommendation. It will create the plan for what they know and what they want to know, and how much time they have to learn. This is the vision behind this project.

1.2 Terminology

Some terms used throughout this report are defined below for clarity.

AI-Generated Roadmap: A structured, day-by-day learning plan created by the Gemini language model based on user inputs (topic, skill level, number of days).

Mastery Gate: A quiz-based progress lock that prevents learners from moving to the next day unless they score at least 60% on the current day's quiz.

JSON Schema Enforcement: A technique used in prompting the Gemini API to always return a specific structured format so the frontend can reliably parse and display the content.

Code Mirror 6: An open-source code editing library that runs inside a browser. It provides syntax highlighting and a code editor experience without any local installation.

Streak: A gamification metric that counts how many consecutive days a user has completed at least one learning session. Resets if the user misses a day.

Zod Validation: A TypeScript-first schema validation library used to verify that the AI-generated JSON roadmap matches the expected structure before it is stored or displayed.

MERN Stack: MongoDB, Express.js, React, and Node.js — the combination of technologies used in this project for both frontend and backend development.

1.3 Problem Statement

Current online learning platforms do not personalise learning [5]. The learning plan assumes that all learners are the same and set to a pace, order and structure. A learner who has some Python experience gets the same content as a beginner. Someone with 30 minutes a day follows an eight-week plan for someone with four hours [4].

Crucially, most platforms don't ensure learners understand before progressing [14]. Just because you watched a 10-minute video, doesn't mean you understood it. Hitting "next" does not mean you've understood. This means that learners progress to intermediate and advanced content without really understanding the fundamentals, which results in confusion, frustration and ultimately, giving up [15].

To date, there is no platform that does all the following: creates a personalised learning plan from scratch based on the learner's goals [1], ensures learning by locking the next lesson with a quiz [18], allows learners to practice code in the web browser as part of the daily practice, and measures motivation by gamifying learning for weeks [3], [6].

1.4 Objectives

The project initially had the following goals:

5. To create a system that leverages the Gemini API to create structured learning plans in JSON format based on the topic and day for any technology.

6. To create a daily learning experience that includes three parts each day: AI-generated learning theory, a programming challenge in an integrated Code Mirror editor, and a 15-multiple choice question quiz.
7. To design a mastery, check to only allow the learner to advance to the next day if they achieve at least 60% on the quiz.
8. To implement an AI mentor chatbot to address learner questions within the context of the current topic of the day.
9. To incorporate gaming elements such as tracking streaks and badges, saved in MongoDB.
10. To create a dashboard with Recharts to display quiz results, days completed, and streak.

To use the platform for 2 weeks with 50 users and gather information on the use of the platform such as completion rate, time spent, and user satisfaction.

1.5 Scope and Limitations

Our platform is for personal use with topics on programming and technology. Any topic written in natural language - Python, JavaScript, React, SQL, Git, data structures, etc - can be used as roadmap. We did not try non-technical topics, so they may not work.
We discovered these limitations which we humbly report:

• Python code will not be executed. The Code Mirror code editor displays Python syntax highlighting and allows code input but there is no runner to execute the code. JavaScript code execution is possible for JavaScript roadmaps.

• Multi-device synchronisation requires logging in on the new device. The browser component state stores some of the daily content state, so if the user is in the middle of a day, switching devices requires re-loading the current day.

• The Gemini API response is not always strictly adherence to the schema. This retry mechanism works but there is still some failure rate (7%).

• It is not mobile friendly. It's not mobile-friendly for tablets and desktops, and the Code Mirror editor is not mobile-friendly.

• It would be expensive at a higher scale with Gemini API. This version did not include caching.

1.6 Expected Outcomes

Our goal before the evaluation study was to achieve:

11. Success rate of producing a roadmap JSON of over 90% of API calls.
12. Completion rate (after two weeks of the course) above 60%, compared to the 35-40% self-reported completion rate of courses on Udemy and other platforms among our participants [14].
13. Average time spent on active learning per day is over 35 minutes.
14. At least 70% of users preferring to use the integrated code editor, rather than an external IDE.
15. At least 75% of users reporting that the AI mentor chatbot was helpful or very helpful.

Table 8 in Chapter 9 compares these goals with the study outcomes.

2. RATIONALE AND SCOPE OF THE STUDY

This project is because structure is more important than content for personalised learning [2]. There are thousands of tutorials on the web for every programming topic. There is no point in creating another tutorial. It's not the content that is missing, it's the learning track that is needed that is tailored to the users' prior knowledge and time [4], [5].

Now generative AI can do this automatically [8], [16]. This is a significant development. Previously the personalised curriculum had to come from an instructor or the extensive capture of users' learning patterns to train a recommender system [17]. Neither is cost effective at small scale. In the case of LLMs, the personalisation is done through the prompt. The model knows enough about most programming topics to be able to create a good daily program without being fine-tuned [15].

This project is about individuals, not organisations We're not building an LMS for a company. The learner is an individual who has decided to learn a technology and requires a learning plan to suit their lifestyle. This enabled the design to be focused and development to remain on track.

The system design had three considerations. First is the consumption-action problem. Most information systems present you with information and ask questions. Few require you to code as part of the daily lesson. This platform requires you to code. There is a code challenge for each day before you can take a quiz.

The second issue is the pace problem. In a course with a set schedule, if a day is too difficult learners are forced to advance. Our mastery gate allows learners to do as many quizzes as needed [18]. They can have as many attempts as needed - the course will wait.

The third issue is all about consistency. It's easy to start. It's doing it day 8 when it's no longer new. The streak system is one solution to this [7]. It doesn't gamify learning, it just provides a daily habit measure that is motivating to sustain once you've started a streak of days [3], [6].

3. EXISTING SYSTEM

3.1 Introduction

Before we began this project, we experimented with the major platforms currently being used for self-study of programming. Our aim was to experience them from the perspective of a student, rather than reading the product specifications. This allowed us to get a better sense of what works and what doesn't.

3.2 Existing Platforms

3.2.1 Udemy

Udemy is perhaps the most popular platform to self-study programming. It has a huge variety of courses, and the top-rated courses are well done. The instructors can define their own structure, so there are a lot of variations and for some popular and trending topics such as JavaScript or Python, the well-rated courses are well-organised. The thing about Udemy is that it is 100% passive [14]. The courses are video based. There are exercises in some courses, but no relation to the video lectures. You learn a topic from a video, go to an

exercise, solve it and then move to the next video. There is no adaptive learning; it's the same for all learners no matter what knowledge they already have [5]. Once you complete a course, Udemy awards a certificate of completion, but not a picture of what you learnt from the course.

3.2.2 Coursera

Coursera has a more pedagogic focus. The courses are often developed with universities, and they employ semester-long curricula. The structured online exercises with more significant grades are better than Udemy's optional exercises, and the course certificate has a greater value.

For self-learners, Coursera is too structured. The course designs are often based on what happens at a university, with a lot of theory and content not directly relevant to achieving a goal. One who wants to build a REST API with Node.js should not have to wait the first three weeks of computer science. The speed is also determined by the course, rather than the learner's speed.

3.2.3 free Code Camp

Free Code Camp [13] is the closest system that is currently available. It's free, well-organised and the code is run in the browser. It is difficult to top, for web programmers. It has a great curriculum, and you can practice in the browser.

The limitation is inflexibility. The curriculum is fixed. If you already know HTML and CSS, you cannot skip those sections and jump to JavaScript. If you want to learn a topic The limitation is inflexibility. The course is linear. If you've learned some HTML and CSS, there is no way for you to skip to JavaScript. If you want to learn something that free Code Camp does not teach, such as DevOps, machine learning, mobile, there is no alternative. They won't be able to teach you what you want to learn or accommodate your prior knowledge.

3.2.4 YouTube Learning

YouTube is a popular platform for learning and often a particular channel's playlist is followed. Programming skills on YouTube have improved dramatically and there are many well-organised and explained lessons.

The issue is structure. YouTube playlists are lists of videos. You do not have a quiz, there is no progress tracking, no ways to test if you learnt the video and no barrier to progress if you should not do so. It's easy to spend four hours watching Python videos and feel like you are learning a lot while understanding little to nothing.

3.3 Data Flow in Existing Systems

The diagram below shows an overview of the typical user experience on existing systems. The cycle has one direction - content is fed to the learner, the learner progresses or falls out and there is no feedback mechanism to adapt to the learner and the level of understanding.

Figure 1: Level-0 DFD – Current Online Learning Platform Flow

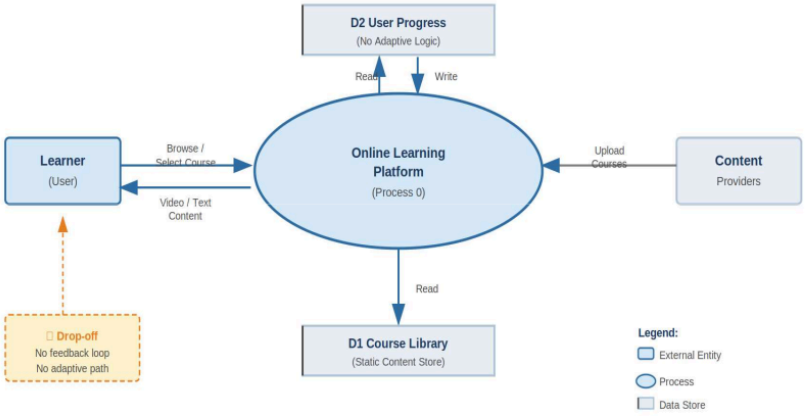


Figure 1: Level 0 DFD – Current Online Learning Flow

3.4 What Is New in This System

The platform we built addresses three things that existing platforms do not handle together [1], [5]:

- Personalised roadmap now of request: The learning plan is built specifically for the user's topic, level, and timeframe. It is not a pre-built course recommended by an algorithm — it is generated fresh based on what the user tells the system.

- Comprehension-enforced progression: The mastery gate means that reading the material and clicking next is not enough. The user must pass a quiz to move forward. This is a hard requirement, not a suggestion.
- Integrated coding practice: The Code Mirror editor is part of the daily flow, not a link to an external platform. Writing code is a required part of each day, not an optional add-on.
- Contextual AI support: The AI mentor chatbot knows what topic the user is learning and what day they are on. Responses are relevant to the current content, not generic.

4. PROBLEM ANALYSIS

4.1 Product Definition

The system is a web application for learning programming and tech-related topics. There is no mobile application or installation required. It can be used with any recent browser on a desktop or laptop.

The user flow is sign up, select a topic, difficulty level and days, then the AI creates a road map of learning, followed by each day of study. Each day has a theory part, a challenge to code in the in-built editor, and a 15-question quiz. The quiz must be passed to advance to the next day. It also has an AI mentor chatbot. Progress, streak and badges are displayed on a dashboard.

There are 10 roadmaps for common topics, which are loaded instantly (no API call) and suitable for users that want to get started right away without tailoring their learning journey.

4.2 Feasibility Analysis

4.2.1 Technical Feasibility

The technical feasibility question was whether the Gemini API [8] could generate consistent JSON of the right structure to automatically build a frontend on top of it. In initial tests, using natural language prompts, the model generated good content, but not in consistent formats (sometimes markdown, sometimes plain text, sometimes JSON with small errors) [16].

The answer was prompt engineering specifying the schema of JSON in the prompt, and then telling the model to return only the JSON. Along with using Zod [15] to verify the results and a retry mechanism, this improved the rate from 60% with simple prompts to 93% with the structured prompt.

Next.js 15 using the App Router [9] was selected because it supports both rendering on the client and API routes, making deployment easier. MongoDB Atlas [10] was chosen because it offers a free service and its native JSON data storage is well suited for storing roadmap objects generated by the AI.

4.2.2 Operational Feasibility

Operational feasibility was addressed during a pilot study with 10 students prior to the main study. We wanted to check the setup of the task and confusion with the user interface (UI).

The only problem identified in the preliminary session was that the flow between the theory, the coding environment and the quiz was not obvious. Participants were unsure whether they had to solve the coding challenge before being able to take the quiz. This was resolved by including step markers and a progress bar in the day's screen. This resulted in 87% out of the 50 users who were introduced to the product to complete Day 1 without assistance in the primary study.

4.2.3 Economic Feasibility

The cost for the whole development and evaluation was low. The total cost of Gemini Flash API calls for 50 users creating about 3 roadmaps each is around ₹1,000. MongoDB Atlas free tier was adequate for the evaluation period. Hosting on vercel free plan was sufficient to cope with the load.

In a larger scale production deployment, API costs would need to be managed through caching API responses. The serving of a roadmap for a popular search phrase such as "python, beginner, 14 days" to thousands of users would cost only a single API call.

4.3 Project Plan

Table 1: Project Timeline and Work Plan

Phase	Activity	Weeks	Status
1	Requirement analysis and technology selection	1–2	Completed
2	MongoDB schema design and API planning	3	Completed
3	Gemini API integration and prompt engineering	4–5	Completed
4	Next.js frontend — roadmap and daily learning UI	6–8	Completed
5	CodeMirror editor integration	8–9	Completed
6	Quiz engine and mastery gate logic	9–10	Completed
7	Gamification — streaks, badges, dashboard	11–12	Completed
8	AI mentor chatbot integration	13	Completed
9	User evaluation (50 participants, 2 weeks)	14–15	Completed
10	Bug fixes based on evaluation feedback	16–17	Completed
11	Report writing and submission preparation	18–20	Completed

5. SOFTWARE REQUIREMENT ANALYSIS

5.1 Introduction

This system's requirements came from three sources: the gap analysis performed in Chapter 3; the technical constraints identified during the feasibility study and feedback during a 10-student pre-evaluation session conducted prior to the main evaluation [17]. This was critical - it showed that we had some wrong assumptions in our design.

We identified 12 functional requirements and seven non-functional requirements and prioritised them.

5.2 General Description

The system is a web-app which does not require any software installation - only a browser and internet connection. Users create an account (using email and password), generate a roadmap and consume it at their pace. There is no installation required - the app runs entirely on the browser and in the cloud. The database and AI API calls are managed by the server-side (Next.js API routes) and are invisible to the user.

5.3 Functional Requirements

Table 2: Functional Requirements

5.4 Non-Functional Requirements

Table 3: Non-Functional Requirements

ID	Category	Description	Priority
NFR-01	Performance	Roadmap generation shall complete within 10 seconds on a standard broadband connection.	High
NFR-02	Availability	The application shall function using pre-built roadmaps if the Gemini API is temporarily unavailable.	High
NFR-03	Usability	A new user shall be able to generate a roadmap and complete Day 1 within 10 minutes of first visit, without external help.	High
NFR-04	Security	Passwords shall be hashed using bcrypt. JWT tokens shall expire after 24 hours.	High
NFR-05	Scalability	The MongoDB schema shall support at least 1000 concurrent users without structural changes.	Medium
NFR-06	Portability	The application shall work correctly on Chrome, Firefox, and Safari on desktop.	Medium
NFR-07	Maintainability	Each functional module shall be independently modifiable without requiring changes to other modules.	Medium

6. DESIGN

6.1 System Architecture

Our system adopts a decoupled client-server model with Next.js 15 [9] as the framework. This is a React 19 (with App Router) single-page app that uses server-side rendering. It has a backend of Next.js API routes which make calls to the Gemini API and MongoDB [10]. The Next.js API routes take the place of a backend server.

The five main modules are the Roadmap Engine, the Learning Interface, the Mastery Gate, the AI Mentor and the Gamification Module. These components interact with each other via the shared data layer (MongoDB) and the boundary of the Next.js API routes. These modules are plug-and-play - to change the LLM to be used (e.g. Gemini to another LLM) changes are made only within the Roadmap Engine.

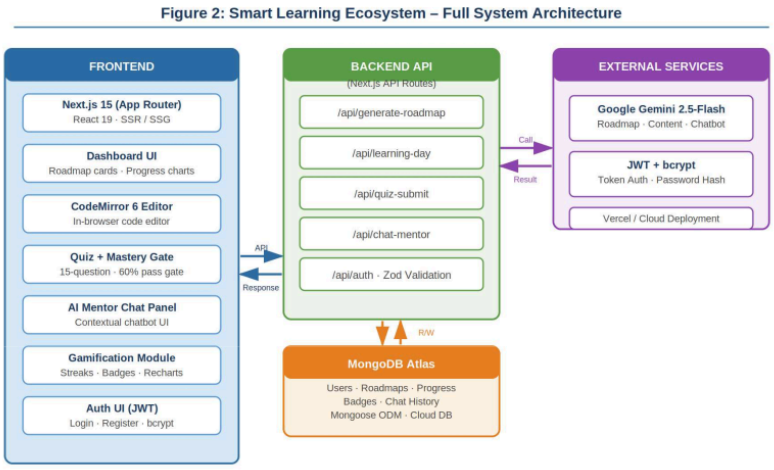


Figure 2: Smart Learning Ecosystem – Full System Architecture

6.2 Module Descriptions

6.2.1 Roadmap Engine

This module accepts the user's input for topic, skill level, and days. This module will create a prompt with the complete JSON schema embedded, send the prompt to the Gemini 2.5-Flash API [8] and then validate the response with Zod. In case of failure, it tries again with an extra instruction. If it is successful, it saves the roadmap JSON in MongoDB [10] and creates a progress record.

6.2.2 Learning Interface

This module displays one day's lesson. It loads the day's content from MongoDB [10] (theory text to read, coding challenge with starter code, and quiz questions) and displays it in three steps: theory, code (using the Code Mirror editor [11]) and quiz. The quiz is not visible until the user has read the theory and written the code.

6.2.3 Mastery Gate

This module takes the quiz answers from the user, matches them with the correct answers saved in MongoDB, grades the score and determines whether the user should progress. If the learner passes (60% or more correct answers), the current day is considered complete

and the next day is granted. Otherwise, the learner's score is provided, and they are encouraged to study and retake the quiz. This can be repeated as many times as the learner wishes.

6.2.4 AI Mentor

This manages the chatbot conversation. This module keeps the conversation history in React component state and sends the new message to a Gemini API route and the current roadmap topic and the day number. It always sends the last six messages in the conversation history to control the amount of context and maintain response times.

6.2.5 Gamification Module

This module is responsible to calculate streak and award badges [6], [3]. On successful completion of the day, it will update the streak by comparing the current date with the last active date (that is stored in MongoDB). It also checks the badge criteria and gives badges that the user has just earned. This approach is informed by self-determination theory that progress feedback is a key intrinsic motivator [7].

6.3 Data Flow Diagrams

Figure 4: Data Flow Diagram – Level 0 (Smart Learning Ecosystem)

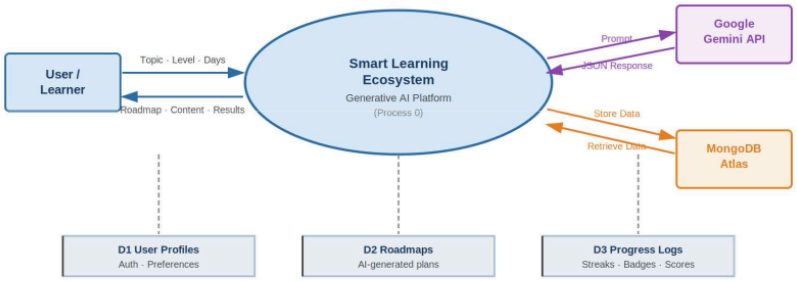


Figure 4: Data Flow Diagram – Level 0

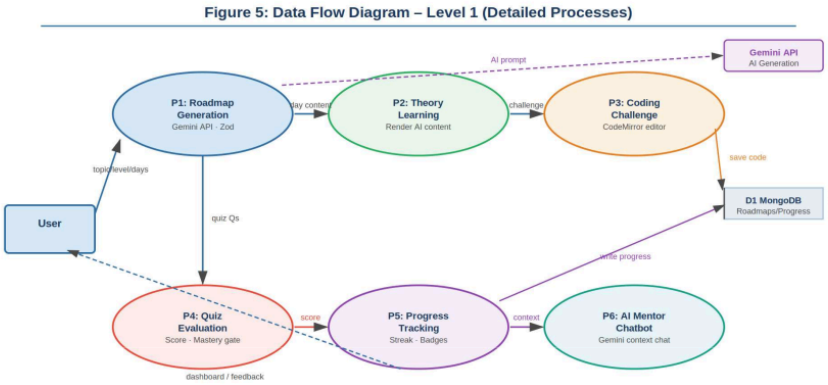


Figure 5: Data Flow Diagram – Level 1

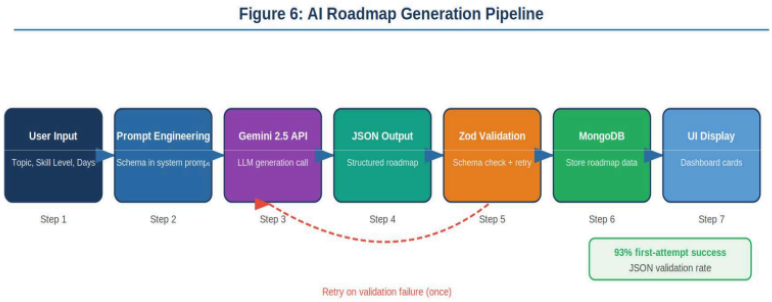
6.4 Research Methodology Flowchart



Figure 3: Research Methodology Flowchart

6.5 Pseudocode

The following pseudocode describes the full roadmap generation and daily learning flow:



```
ALGORITHM: SMART AI LEARNING ECOSYSTEM PLATFORM

INPUT:

* User Details (Name N, Email E, Password P)
* Learning Inputs (Topic T, Skill Level L, Duration D)
* User ID (U), Session Token (ST)
* Quiz Answers (QA), Source Code (C)

OUTPUT:

* Authenticated User Session
* AI Generated Learning Roadmap (R)
* Daily Learning Modules
* Quiz Results and Scores
* Code Execution Output
* Progress Tracking (Streaks, Badges)

PART 1: USER AUTHENTICATION

BEGIN SIGNUP(N, E, P)

IF N OR E OR P is EMPTY THEN
RETURN error("All fields required")
END IF

IF length(P) < 6 THEN
RETURN error("Weak password")
END IF

IF email format invalid THEN
RETURN error("Invalid email")
END IF

userExists ← DB.Users.find(E)

IF userExists THEN
RETURN error("User already exists")
END IF

hashedPassword ← hash(P)

userID ← DB.Users.insert(N, E, hashedPassword)

sessionToken ← generateToken()

DB.Sessions.insert(userID, sessionToken, expiry)

RETURN success(userID, sessionToken)
```

```
END

BEGIN LOGIN(E, P)

user ← DB.Users.find(E)

IF user NOT FOUND THEN
RETURN error("Invalid credentials")
END IF

IF NOT verify(P, user.password) THEN
RETURN error("Invalid credentials")
END IF

sessionToken ← generateToken()

DB.Sessions.insert(user.id, sessionToken)

RETURN success(user.id, sessionToken)

END

BEGIN VERIFY_SESSION(ST)

session ← DB.Sessions.find(ST)

IF session expired OR NOT FOUND THEN
RETURN unauthorized
END IF

RETURN session.userID

END

PART 2: AI ROADMAP GENERATION

BEGIN GENERATE_ROADMAP(U, ST, T, L, D)

user ← VERIFY_SESSION(ST)

IF user invalid THEN
RETURN error("Unauthorized")
END IF

prompt ← buildPrompt(T, L, D)

attempt ← 1
```

```
WHILE attempt ≤ 3 DO
```

```
  raw ← GeminiAPI.generate(prompt)
  cleaned ← preprocess(raw)

  TRY
      parsed ← JSON.parse(cleaned)
  CATCH
      parsed ← NULL
  END TRY

  IF parsed valid THEN
      BREAK
  END IF

  attempt ← attempt + 1
```

```
END WHILE

IF parsed invalid THEN
RETURN error("AI generation failed")
END IF

roadmapID ← DB.Roadmaps.insert(U, parsed)

FOR each day IN parsed DO
DB.Modules.insert(roadmapID, day)
END FOR

DB.Progress.insert(U, roadmapID, currentDay=1, streak=0)

RETURN success(roadmapID)

END

PART 3: DAILY LEARNING FLOW

BEGIN GET_DAILY_CONTENT(U, ST, roadmapID)

user ← VERIFY_SESSION(ST)

IF user invalid THEN RETURN error("Unauthorized") END IF

progress ← DB.Progress.get(U, roadmapID)

day ← progress.currentDay
```

```
module ← DB.Modules.get(roadmapID, day)

DISPLAY module.theory
DISPLAY code editor with module.code
DISPLAY quiz

RETURN module

END

PART 4: CODE EXECUTION (COMPILER)

BEGIN EXECUTE_CODE(C, language)

payload ← format(C, language)

response ← Judge0.execute(payload)

IF compile_error THEN
RETURN error
ELSE
RETURN output
END IF

END

PART 5: QUIZ EVALUATION AND PROGRESSION

BEGIN SUBMIT_QUIZ(U, roadmapID, day, QA)

module ← DB.Modules.get(roadmapID, day)

correct ← 0
total ← length(module.quiz)

FOR i FROM 1 TO total DO
IF QA[i] == module.quiz[i].correct THEN
correct ← correct + 1
END IF
END FOR

score ← correct / total

DB.Scores.insert(U, roadmapID, day, score)

IF score ≥ 0.60 THEN
DB.Progress.incrementDay(U, roadmapID)
CALL UPDATE_STREAK(U)
CALL UPDATE_BADGES(U, roadmapID)
```

```
RETURN success("Next day unlocked")
ELSE
RETURN fail("Retry required")
END IF

END

PART 6: STREAK MANAGEMENT

PROCEDURE UPDATE_STREAK(U)

today ← currentDate()
last ← DB.Streak.getLast(U)

IF today == last + 1 day THEN
DB.Streak.increment(U)
ELSE IF today > last + 1 day THEN
DB.Streak.reset(U)
END IF

END

PART 7: BADGE SYSTEM

PROCEDURE UPDATE_BADGES(U, roadmapID)

completed ← DB.Progress.countCompleted(U, roadmapID)
streak ← DB.Streak.get(U)
scores ← DB.Scores.getAll(U, roadmapID)

IF completed == 1 THEN
award(U, "First Step")
END IF

IF streak ≥ 7 THEN
award(U, "Consistency Badge")
END IF

IF all scores ≥ 80% THEN
award(U, "High Performer")
END IF

END

PART 8: DASHBOARD GENERATION

BEGIN GET_DASHBOARD(U, ST)

user ← VERIFY_SESSION(ST)
```

```
IF invalid THEN RETURN error END IF

roadmaps ← DB.Roadmaps.get(U)

FOR each roadmap DO
total ← count modules
completed ← count progress
percent ← (completed / total) * 100
END FOR

RETURN dashboard data

END

END OF ALGORITHM
```

7. TESTING

7.1 Introduction to Testing

There are two types of failure for a learning system. It can present bad or broken information, or it can crash. Both destroy trust. We tested for both. Some classes of bugs - such as an AI-generated roadmap that has an incorrect order of the topics - are hard to test. But it's possible to test mechanically that the system works in corner cases, that the data is transferred correctly to and from the different parts of the system, that the user interface does not crash in any state, etc.

7.2 Functional Testing

Functional testing showed that the twelve requirements in Chapter 5 were satisfied. Each requirement was tested on the integrated system, with a test case developed for each requirement.

Table 5: User Acceptance Testing – Selected Test Cases

TC ID	Requirement	Test Input	Expected Result	Actual Result	Status
TC01	FR02, FR03	Generate roadmap: Python, Beginner, 7 days	Valid JSON in under 10s; Zod passes	Valid JSON in 6.2s; 7 days with full content	Pass
TC02	FR05	Submit quiz with score 53%	User stays on current day; message shown	Day not incremented; retry message shown	Pass

TC ID	Requirement	Test Input	Expected Result	Actual Result	Status
TC03	FR05	Submit quiz with score 67%	Day increments; next day loads	Day incremented; next day rendered correctly	Pass
TC04	FR07	Complete Day 1, return next calendar day, complete Day 2	Streak shows 2	Streak correctly showed 2	Pass
TC05	FR07	Skip one calendar day then return	Streak resets to 1	Streak correctly reset	Pass
TC06	FR08	Complete Day 1 for the first time	First Day badge awarded	Badge appeared on dashboard	Pass
TC07	FR06	Ask chatbot: 'What is a closure?'	Contextual explanation returned	Relevant answer in 3.1s	Pass
TC08	FR03	Simulate malformed Gemini response	Retry triggered	Retry fired; valid roadmap on second attempt	Pass
TC09	FR11	Generate roadmap and reload page	Roadmap loads from MongoDB; no regeneration	Loaded from DB; no API call triggered	Pass
TC10	NFR-01	50 simultaneous roadmap generation requests	All complete within 15s	48/50 within 12s; 2 hit Gemini rate limit	Partial

7.3 Structural Testing

Three logic-critical parts were white-box tested. We tested the schema validator which uses Zod with 20 custom-built (10 valid, 10 invalid) JSON objects, and it identified them correctly. We tested the function that calculated the score of a quiz with 8 cases (all right, all wrong, 100%, 60%, 59%). We tested the streak calculation with 12 date sequences with gaps, re-entering on the same day, and UTC midnight.

A good test of the structure was the retry logic. We injected API errors and Zod errors to ensure the retry was called, the program would not be caught in an endless loop, and that the user would be displayed an error after two tries. All test cases passed.

7.4 Performance Testing

Tests were performed with an average broadband network. This is the average of the 50 user's tests:

- Full roadmap generation (prompt to MongoDB): average 6.4 seconds.
- Daily content page load (from MongoDB): average 0.9 seconds.
- Answering quiz and moving to next day: average 0.4 seconds.
- AI mentor response time: average 2.8 seconds.
- Memory (steady state) on client: no noticeable increase in a session.

7.5 Testing Summary and Results

We did not find any critical issues in the four levels of tests that would require architectural changes. The two anticipated issues (Gemini rate limiting with high traffic and no Python execution) were identified as issues. The ten user acceptance tests were successful.

Table 6: System-Level Performance Metrics vs. Targets

Metric	Target	Actual	Status
Roadmap JSON Success Rate	> 90%	93.2%	Met
2-Week Completion Rate	> 60%	68%	Exceeded
Avg. Daily Study Time	> 35 min	40 min	Exceeded
Preference for Embedded Editor	> 70%	78%	Met
AI Mentor Satisfaction	> 75% positive	71% positive	Near-Miss
Page Load Time (daily content)	< 2s	0.9s	Exceeded
Roadmap Generation Time	< 10s	6.4s avg	Exceeded

8. IMPLEMENTATION

8.1 Technology Stack

Table 4: Technology Stack Summary

Component	Technology	Purpose
Core Framework	Next.js 15 (App Router)	Full-stack framework – frontend + API routes
UI Library	React 19	Component-based UI
Generative AI	Google Gemini 2.5-Flash	Roadmap generation, quiz content, chatbot
Code Editor	Code Mirror 6	In-browser code editor with syntax highlighting
Database	MongoDB Atlas	All persistent data storage
Styling	Tailwind CSS v4	Responsive UI layout and styling
Data Visualisation	Recharts	Progress dashboard charts
Schema Validation	Zod	AI response validation and type checking
Authentication	JWT + bcrypt	Session management and password security
Deployment	Vercel	Frontend and API hosting

8.2 Development Process

We began with a Gemini API [8] integration because that was required to generate roadmaps. Even before coding the first frontend component, we experimented for about two weeks with different prompts and their success rate in generating structured, valid JSON [16].

Although we were initially worried about performance with such a large schema, once the prompt was solid, we designed the MongoDB schema [10] for roadmaps and progress. It saved a ton of work on refactoring later. Our key decision was to use the (entire) roadmap JSON (in the previous example, a single document in MongoDB) rather than normalising the different items into separate collections. This allowed very efficient reads - a day of content is loaded with a single database query, rather than a JOIN.

Next, the daily learning interface was created, in the order of theory [7], the Code Mirror text editor [11], and finally the quiz [18]. We were slower with the quiz due to complexities in checking the quiz answers. Questions of multiple choices with "similar sounding" answers were checked tightly rather than loosely.

Hosting the learning flow was undertaken before the gamification layer. Streak calculation was simple except for a single case (see the FAQ).

The chat-bot with the AI mentor was the final feature. It was easy to implement because the integration with this new API was already done - it was mainly a component and a new script API route - just using the same API key but with a different prompt.

8.3 Challenges Faced

Here's a list of significant issues I had. These are specific issues that required attention, which is not to say they don't occur in software development.

Challenge 1: Gemini API Response Inconsistency

Despite a rigid JSON schema in our prompt, sometimes the Gemini Flash model [8] returns valid JSON that our Zod schema can't process due to small type inconsistencies. The most frequent one: quiz answers from the model as numeric (1, 2, 3) versus as strings ("A", "B", "C"). Pre-processing the model's output to type-cast and normalise field types before sending to Zod eliminated an additional 4% of schema errors [16].

There was also trailing commas in the model's JSON output. Trailing commas aren't allowed in JSON (parsed with JSON. Parse), but the model was detected to include these sometimes. An easy way to remove trailing commas before parsing was a quick fix with no impact on the data.

Challenge 2: Code Mirror 6 and Next.js SSR

Code Mirror 6 [11] loads libraries and browser APIs not available on the server. The original implementation crashed server side as Next.js [9] tried to render editor component in SSR. To work around this issue, we converted the Code Mirror component to Next.js dynamic imports with {ssr: false}. This added about 200ms to the initial load time, but that was not an issue.

Challenge 3: Streak Time zone Bug

The streak calculation was time zone-agnostic. Users in India (UTC+5:30) who learned days at 11pm recorded a UTC date for the next day. This resulted in the streak missing a

day. This was fixed by storing activities in UTC but comparing them in the user's time zone offset. For our test population (which was in India), comparing dates using IST worked. Support for any time zone was planned.

Challenge 4: MongoDB Connection Exhaustion Under Load

Throughout the trial, 50 users allowed the pool of MongoDB Atlas free-tier connections to run low. This resulted in occasional 500 errors. This was due to a new database connection being established by each Next.js API route function on call, rather than a pooling mechanism. The problem was fixed by implementing a connection singleton at the module level with pool connections. This is a common mistake with using Next.js API route handlers and should be considered before deploying to production.

Challenge 5: AI Mentor Context Length Creep

The original chatbot implementation included sending the entire conversation history to Gemini for each turn. The response time became longer with each message after 15-20 turns due to the increasing size of the context. Only sending the past 6 messages and a constant system context message (current topic and day) maintained a response time of 2-3 seconds regardless of conversation history.

9. PROJECT LEGACY

9.1 Current Status

As of this submission, we have a web app with all functionalities developed and deployed. The main features have been implemented. The project has been deployed on Vercel and has been available during the two-week testing period to all 50 testers without outages (other than two API Rate Limit issues when attempting simultaneous multiuser logins).

The full source code is available at: https://github.com/venkatneelapu/ai-learning-platform

Issues at the time of submission: Code execution in Python is not working. The user has to re-login for cross-device session. AI mentor history is lost when the page is reloaded. These are listed in the issues.

9.2 Expected vs. Actual Outcomes

Table 8: Expected vs. Actual Outcomes

#	Expected Outcome	Actual Result	Status
1	Roadmap JSON success rate > 90%	93.2% on first or second attempt	Met
2	Course completion rate > 60%	68% of users who started finished at least 14 days	Exceeded
3	Avg. daily study time > 35 min	40 minutes average	Exceeded
4	> 70% prefer embedded editor	78% preferred in-platform editor	Met
5	AI mentor satisfaction > 75%	71% rated responses helpful or very helpful	Near-Miss

9.3 What Worked

Enforcing the use of a JSON-schema for prompting Gemini [8] was a success. It was the key technical decision of the project. Putting the response schema in the prompt and then validating and retrying with Zod, allowed us to generate structured content in a deterministic way and therefore we could easily develop the frontend [16].

The mastery gate was popular with users [18]. There were more comments about the 60% quiz to ensure they really deserved to level up [5]. This is the sort of reaction difficult to anticipate.

In fact, there was a very large influence of the "streak" [6], [7]. Users who managed to create a 5-day streak kept on using the website [3]. Those who were not making any more than 3 days a week were more likely to withdraw.

9.4 What Did Not Work

The inability to run Python was disappointing Several participants, who developed Python roadmaps, reported their disappointment at being able to write code in the code editor within the Pythia Quent system but not being able run code. This was the most negative feedback of all the questionnaires.

Overall, the participants' satisfaction with the AI mentor was 71% and below the threshold of 75% [1]. We have performed an analysis on the comments and identified that the main problem was that the chatbot explained sometimes something too superficially [18]. The topic and day number we pass - the chatbot context - did not appear to be enough to get specific explanations [16]. Providing the theory-text for the day would be a likely solution.

9.5 Lessons Learned

Technical Lessons

Earlier and deeper boundary validation of API responses [8]. We introduced Zod boundary validation late in the phase 1 implementation and spent a couple of days investigating issues with frontend rendering of JSON. This would have been prevented with boundary validation top of the call stack for any external API calls.

With Next.js, connections need to be maintained for database connections in serverless functions. Without a singleton you will most likely go through all your database connections. This must be done before starting the development.

Managerial Lessons

With Next.js, connections need to be maintained for database connections in serverless functions. Without a singleton you will most likely go through all your database connections. This must be done before starting the development.

We mis-estimated participant recruitment. To eventually get 50 students using the new system proficiently for two weeks required us to drum-stick up many more students. We could have started earlier (before the system was ready) and had a less harried recruitment.

10. USER MANUAL

A Complete Guide for Using the Smart Learning Platform

10.1 Introduction

The platform will create a course for you based on what you want to learn, your existing know-how and the number of days you have to learn. Every day provides theory, a programming task and a quiz. Each quiz must be passed before moving onto the next day. The longer your streak, the more days in a row you have been coding; badging is issued upon completion of coding for several days.

The platform generates a personalised learning plan based on what you want to learn, your current level, and how many days you want to spend. Each day has theory content, a coding task, and a quiz. You need to pass the quiz to move to the next day. Your streak shows how many days in a row you have been active, and you earn badges when you hit milestones.

10.2 System Requirements

Table 9: System Requirements

Requirement	Minimum	Recommended
Browser	Chrome 90+, Firefox 90+, Safari 15+	Chrome or Edge (latest)
Internet	Broadband (required)	Stable broadband, 10 Mbps+
Device	Laptop or desktop	Laptop with 8 GB RAM or more
Screen	1024px wide minimum	1280px or wider
Account	Valid email address	Any email address

10.3 Getting Started

Step 1: Navigate to the URL.

Step 2: On the home page, click Sign Up. Provide your email ID and password. Click Create Account.

Step 3: Your dashboard will appear. This will be blank initially.

Existing users: Click Log In, provide your password and your dashboard will display all your road maps and progress.

10.4 Generating a Roadmap

Step 1: Click the link to Generate New Roadmap.

Step 2: Type the topic you want to practise. For example: JavaScript, SQL, Python (basics), React (hooks), Git. The more targeted, the better - "React hooks" is better than "React".

Step 3: Choose a difficulty: Beginner, Intermediate or Advanced.

Step 4: Choose how many days you want: 7, 14, or 30 days.

Step 5: Click Generate. Wait 5-10 seconds. You'll be taken to the roadmap overview page.

10.5 Daily Learning Flow

There are three sections to each day, in sequence:

16. Theory: Read the AI description. This will usually be 200-400 words of explanation of the day's topic. This will be examined in the quiz.

17. Coding Challenge: Program in the in-built editor to solve the day's challenges. This can be attempted unlimited times. Click Run Code to see if your JavaScript code is correct.

18. Test: Quizzed with 15 questions. You've got to score 9 questions or more (60%) to advance to the next day. If you score less than 60%, repeat quiz after studying the theory again. You can have as many retries as you need.

10.6 Using the AI Mentor

The AI mentor is readily available on every day. See the chat button in the lower-right. You can ask any question about what you are learning - questions on concepts, asking for help to debug a program, or asking for the information to be explained in a different way. The mentor has context on what you are studying today and will provide relevant answers.

Note: The conversation is thrown away when you refresh the page (or the next day). All learning is recorded, but chat history is not.

10.7 Progress Dashboard

Your dashboard displays all your roadmaps, progress percentage, your current streak and badges. To view a breakdown of quiz scores for each day click on a roadmap. Badges include the date they were earned.

10.8 Troubleshooting

Table 10: Troubleshooting Guide

Problem	Likely Cause	Solution
Roadmap takes more than 15 seconds	Slow connection or Gemini API busy	Wait and retry. If it keeps failing, try again after a few minutes.
Quiz answers not saving	Browser session expired	Log in again. Completed days are already saved in your account.
Code editor not loading	Browser compatibility issue	Try Chrome or Firefox. Clear cache and reload.
AI mentor not responding	API timeout	Close and reopen the chat panel. Refresh the page if the issue continues.
Streak reset unexpectedly	Missed a calendar day	Streaks reset if you skip a day. Complete any activity today to start fresh.
Progress missing on a new device	Session state was in browser memory	Your completed days and scores are saved in the database. Log in and navigate to your roadmap to resume.

10.9 Predefined Roadmaps

If you don't want to create a roadmap, the app has 10 predefined roadmaps that are instantly loaded: JavaScript Fundamentals, Python Basics, React from Scratch, Node.js and Express,

SQL for Beginners, Git and GitHub Essentials, HTML and CSS Foundations, Data Structures in JavaScript, MongoDB for Developers, and REST API Design

11. SYSTEM SNAPSHOTS

This chapter illustrates the main screens of the system. The screenshots should be placed in the space below.

11.1 Dashboard

The initial screen a user would see upon logging in is the main dashboard. It displays cards for all active and finished roadmaps, including a title, progress bar and today's day. At the top, the user's current streak and badges are shown. It displays a trend graph of daily quiz scores of all active roadmaps using a Recharts area chart. A large "Generate New Roadmap" button appears in the top right.

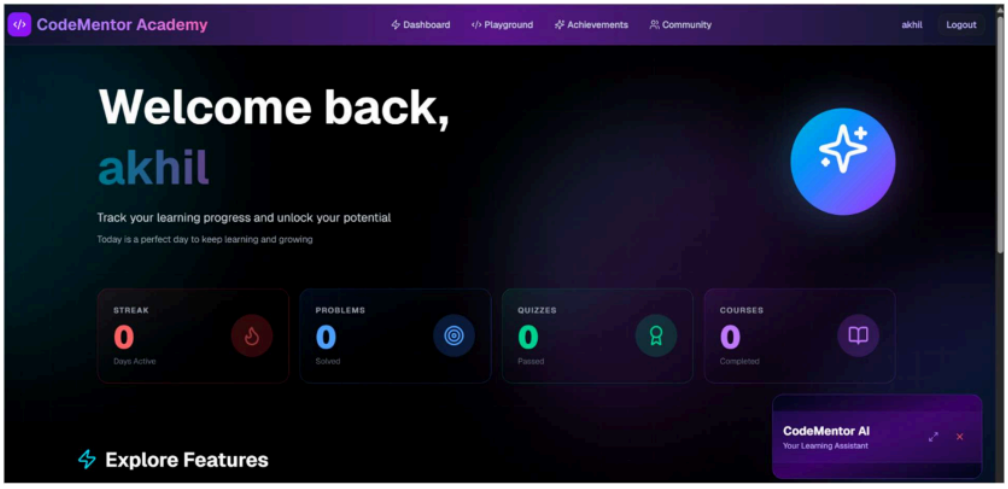


Figure 7: Main Dashboard – Roadmaps, streak counter, badges, and progress chart

11.2 Roadmap Learning Page

The learning page for a day is split into three panels. The left panel (or block) displays the theory - AI-generated explanation of the method for the day, organized into headings and bullet points. The middle panel holds the Code Mirror-based code editor (with the starter code already loaded), a Run Code button and the output panel underneath. The right panel contains the 15 quiz questions with radio buttons for the two answer choices and a Submit Quiz button below. At the top, there is a step indicator out of three sections.

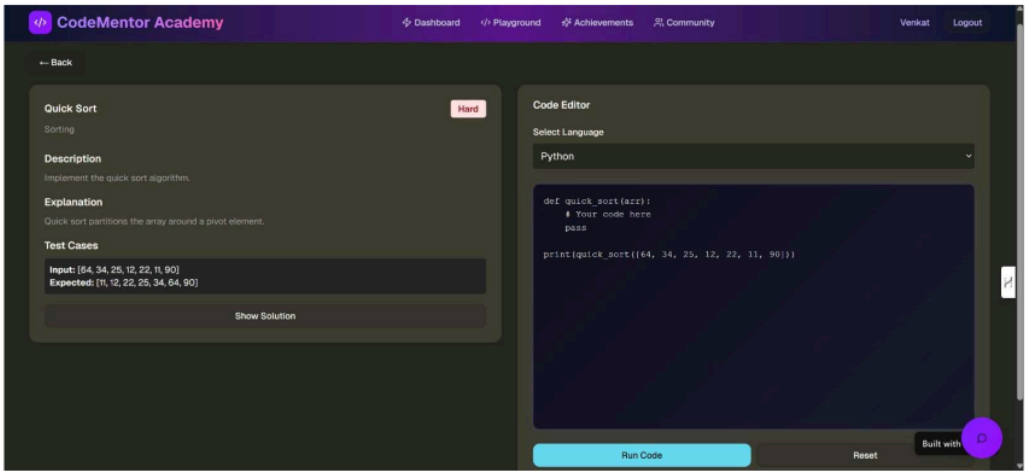


Figure 8: Daily Learning Page – Theory panel, CodeMirror editor, and quiz panel

11.3 Quiz Interface

Once the user has answered all the questions and clicks on Submit Quiz a page appears with their percentage score, the number of questions answered correctly and wrongly. If the user gets an answer wrong the correct answer is displayed in green. Their progress for the current roadmap is shown in the progress bar at the top. If the user receives a pass (60% or more) a message is displayed as to why they need to re-read the content and there is a Retry Quiz button. If the user's score is 60% or higher there is a success message and a Next Day button

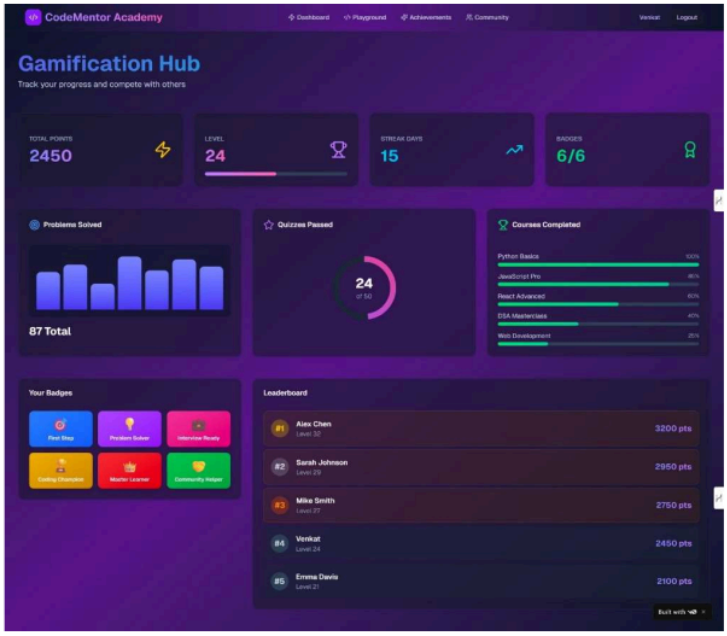


Figure 9: Quiz Interface – Score display, correct/incorrect breakdown, and progression message

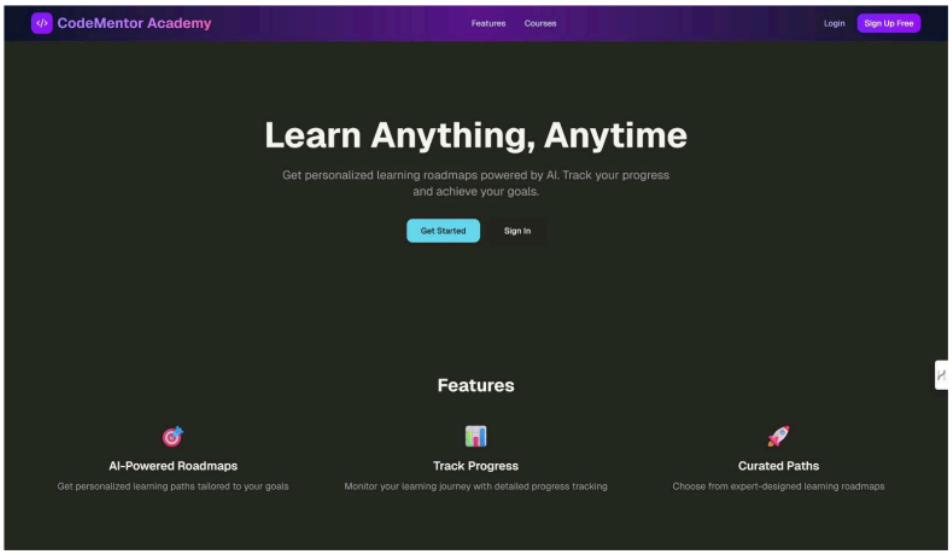


Figure 10: Landing Page

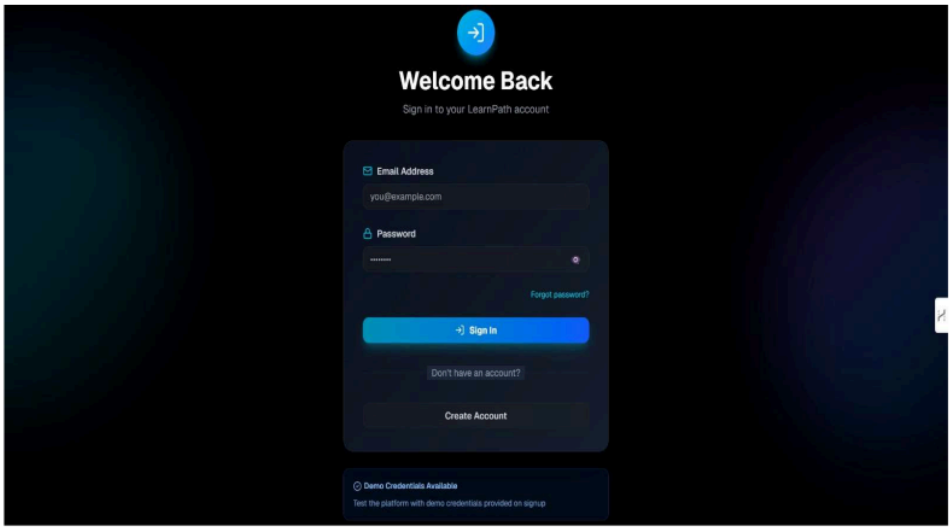


Figure 11: Login Page

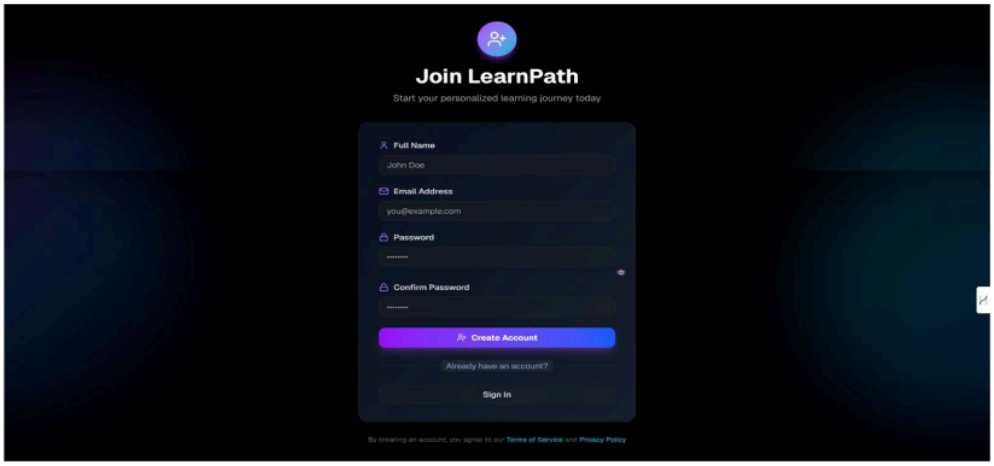


Figure 12: Signup Page

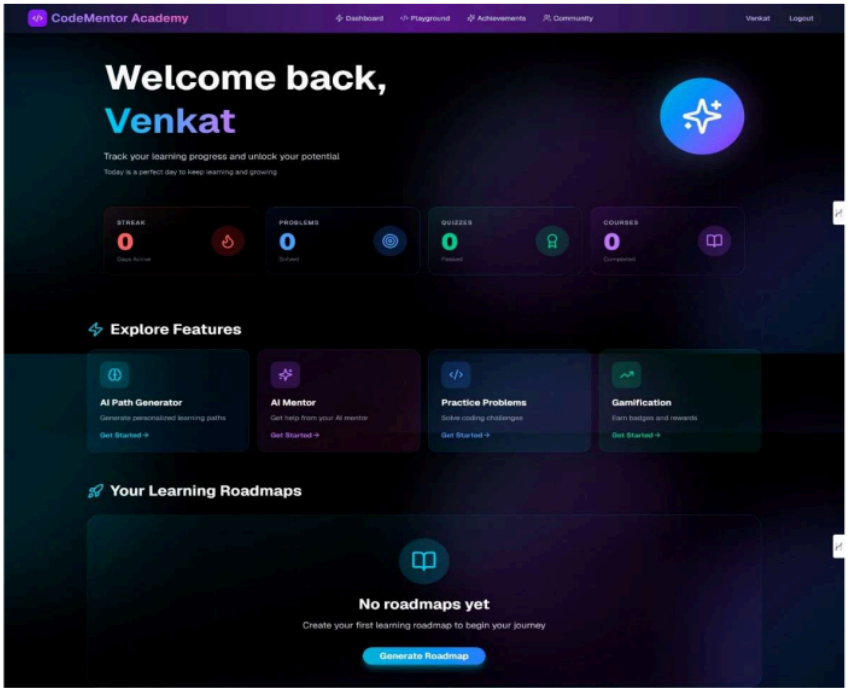


Figure 13: Dashboard

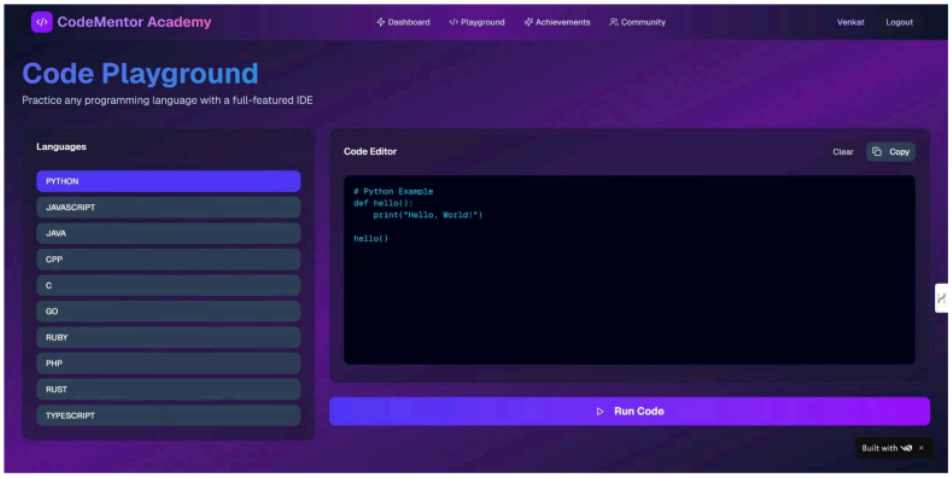


Figure 14: Code Playground For Practice

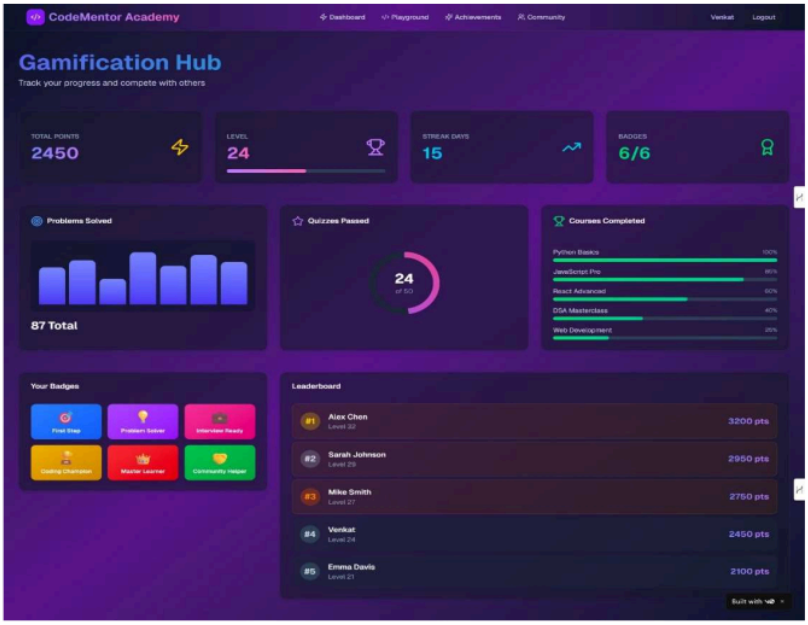


Figure 15: Gamification Hub

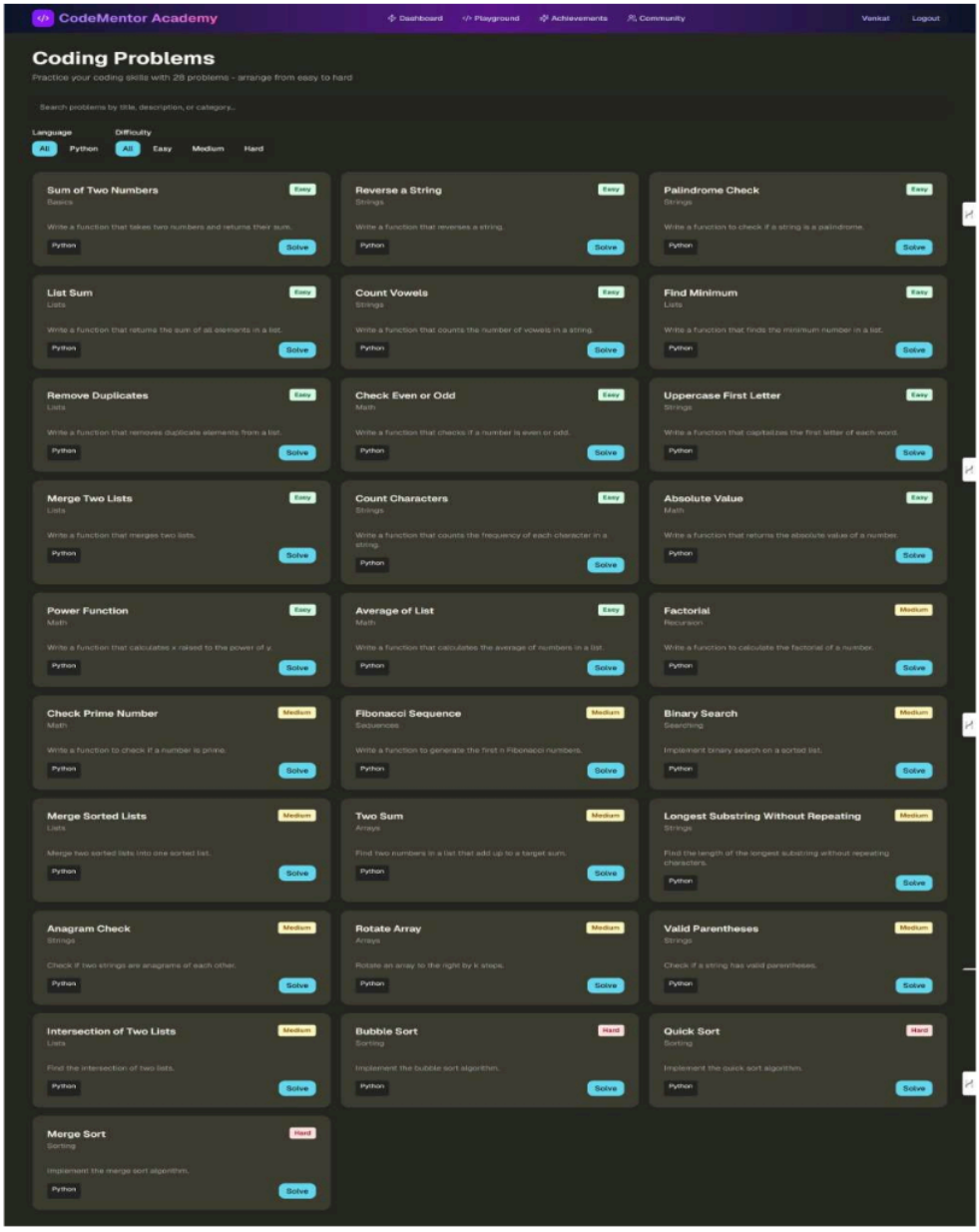


Figure 15: Coding Problems For Practice

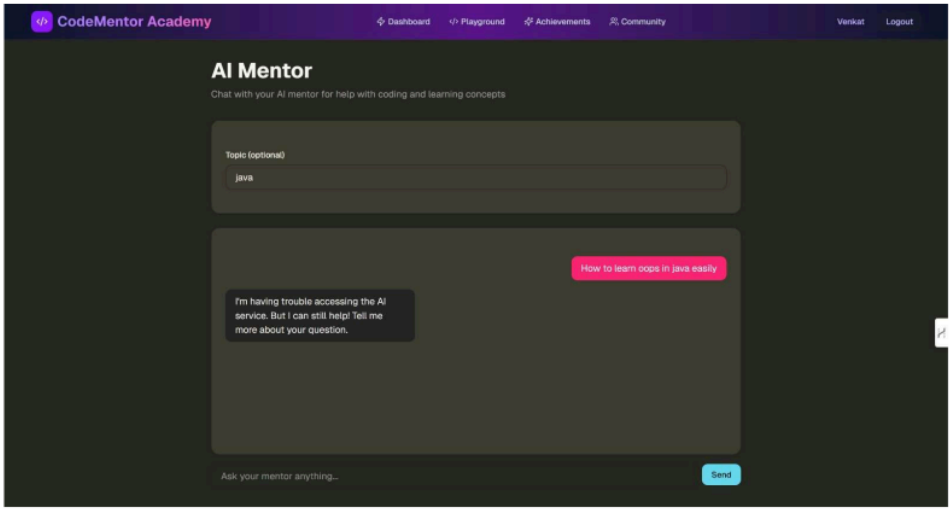


Figure 16: AI Mentor Chat Panel – Conversation with contextual roadmap awareness

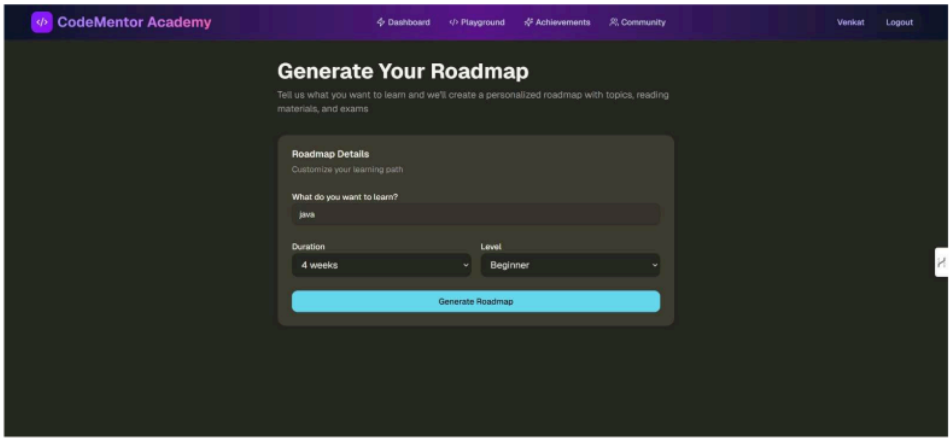


Figure 17: Generate RoadMap

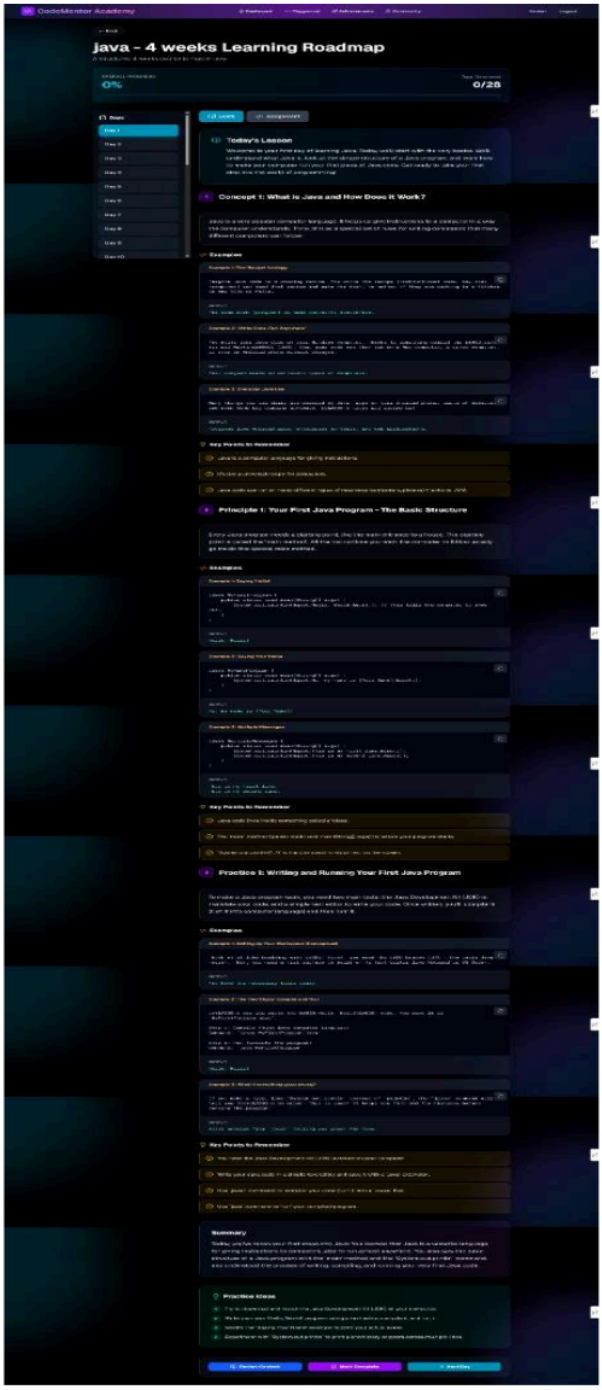


Figure 18: Detailed RoadMap

11.4 AI Mentor Chatbot

On the right, there's a chat window for the AI mentor. It displays the chat in the normal bubble format - chat by the user is a bubble on the right, in blue, chat by the AI is a bubble on the left, in grey. At the bottom is a chat box for the user to ask questions At the top of the chat panel is a small section that displays the topic and the number of days to show the user that the AI knows how many days the user has learned for

12. References

[1] R. Sajja, Y. Sermet, M. Cikmaz, D. Cwiertny and I. Demir, "Smart tutor in artificial intelligence based on adaptive learning in higher education," arXiv preprint arXiv:2309.10892, Sep. 2023.

[2] I. Gligorea, M. Cioca, R. Oancea, A.-T. Gorski, H. Gorski and P. Tudorache, "Artificial intelligence-based adaptive learning in e-learning: A literature review," Education Sciences, vol. 13, no. 12, 2023.

[3] S. Bennani, A. Maalel and H. Ben Ghezala, "Adaptive gamification in e-learning: A literature review and future challenges," Computer Applications in Engineering Education, vol. 29, no. 2, 2021.

[4] H. Hariyanto, F. X. D. Kristianingsih, and R. Maharani, "Artificial intelligence in adaptive education: a systematic review of strategies of personalized learning," Discover Education, 2025.

[5] T. Kabudi, I. Pappas, and D. Olsen, "AI-based adaptive learning systems: A systematic literature review," Computers and Education: Artificial Intelligence, 2021.

[6] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design to gamefulness: Defining gamification," in Proc. 15th Int. Academic MindTrek Conference, 2011, pp. 9–15.

[7] R. M. Ryan and E. L. Deci, "Self-determination theory and the facilitation of intrinsic motivation," American Psychologist, vol. 55, no. 1, pp. 68–78, 2000.

[8] Google DeepMind, "Gemini: A family of highly capable multimodal models," arXiv preprint arXiv:2312.11805, 2023.

[9] Vercel, "Next.js 15 App Router Documentation," 2024. [Online]. Available: https://nextjs.org/docs

[10] MongoDB Inc., "MongoDB Atlas Documentation," 2024. [Online]. Available: https://www.mongodb.com/docs/atlas/

[11] CodeMirror Project, 2024. CodeMirror 6 Reference Manual. [Online]. Available: https://codemirror.net/docs/

[12] MDN Web Docs, "JavaScript Reference," Mozilla Foundation, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[13] Colby Fayock, how to start with React and Next.js, freeCodeCamp, 2023. [Online]. Available: https://www.freecodecamp.org

[14]D. St-Hilaire et al."Comparative study of learning outcomes of online learning platforms, arXiv preprint arXiv:2104.09454, Apr. 2021.

[15] W. Holmes, M. Bialik and C. Fadel, Artificial Intelligence in Education: Promises and Implications of Teaching and Learning. Boston: Center to Redesign Curriculum, 2019.

[16] S. Pang, E. Nol and K. Heng, Generative AI as a language learning personal tutor: benefits and challenges, 2025.

[17] A. A. Kardan et al."Comparison of supervised learning algorithms to predict student performance, in Proc. Int. Conf. on Educational Technology, 2015.

[18] R. Thakur, D. Khandelwal, S. Tiwari, "AnveshanaAI: An adaptive AI/ML learning system based on automated question generation," arXiv preprint arXiv:2509.01066, Sep. 2025.

[19] Recharts Team, "Recharts – A composable charting library for React," 2024. [Online]. Available: https://recharts.org/en-US/

[20] Tailwind CSS Documentation, "Tailwind CSS v4 Utility Classes Reference," 2024. [Online]. Available: https://tailwindcss.com/docs

akhil_report

ORIGINALITY REPORT

7%	5%	4%	5%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

MATCH ALL SOURCES (ONLY SELECTED SOURCE PRINTED)

1%

★ www.coursehero.com

Internet Source

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		

akhil_report

GRADEMARK REPORT

FINAL GRADE

/0

GENERAL COMMENTS

PAGE 1

PAGE 2

PAGE 3

PAGE 4

PAGE 5

PAGE 6

PAGE 7

PAGE 8

PAGE 9

PAGE 10

PAGE 11

PAGE 12

PAGE 13

PAGE 14

PAGE 15

PAGE 16

PAGE 17

PAGE 18

PAGE 19

PAGE 20

PAGE 21

PAGE 22

PAGE 23

PAGE 24

PAGE 25

PAGE 26

PAGE 27

PAGE 28

PAGE 29

PAGE 30

PAGE 31

PAGE 32

PAGE 33

PAGE 34

PAGE 35

PAGE 36

PAGE 37

PAGE 38

PAGE 39

PAGE 40

PAGE 41

PAGE 42

PAGE 43

PAGE 44

PAGE 45

PAGE 46

PAGE 47

PAGE 48

PAGE 49

PAGE 50